

```
In [1]: # This Python 3 environment comes with many helpful analytics libraries installed
# It is defined by the kaggle/python Docker image: https://github.com/kaggle/docker
# For example, here's several helpful packages to load

import numpy as np # linear algebra
import pandas as pd # data processing, CSV file I/O (e.g. pd.read_csv)

# Input data files are available in the read-only "../input/" directory
# For example, running this (by clicking run or pressing Shift+Enter) will list all

import os
# for dirname, _, filenames in os.walk('/kaggle/input'):
#     for filename in filenames:
#         print(os.path.join(dirname, filename))

# You can write up to 20GB to the current directory (/kaggle/working/) that gets pr
# You can also write temporary files to /kaggle/temp/, but they won't be saved outs
```

Importing Modules

```
In [3]: import os
import numpy as np
import re
#import cv2
import matplotlib.pyplot as plt
import tensorflow as tf
import time
from sklearn.model_selection import train_test_split
import os
import matplotlib.pyplot as plt
from PIL import Image
import os
import numpy as np
import matplotlib.pyplot as plt
from PIL import Image
import tensorflow as tf
from tensorflow.keras.layers import Input, Conv2D, MaxPooling2D, Conv2DTranspose, c
from tensorflow.keras.models import Model
from tensorflow.keras.callbacks import EarlyStopping, ModelCheckpoint, ReduceLROnPl
from sklearn.model_selection import train_test_split
```

Getting Basic Stats

```
In [4]: # Path to the dataset directories
image_path = '/kaggle/input/butterfly-dataset/leedsbutterfly/images'
mask_path = '/kaggle/input/butterfly-dataset/leedsbutterfly/segmentations'
```

```
In [5]: # List images and masks
images = os.listdir(image_path)
```

```

masks = os.listdir(mask_path)
# Display data statistics
print(f"The Data statistics\n images: {len(images)} masks: {len(masks)}")

```

FileNotFoundError Traceback (most recent call last)

Cell In[5], line 2

```

1 # List images and masks
----> 2 images = os.listdir(image_path)
3 masks = os.listdir(mask_path)
4 # Display data statistics

```

FileNotFoundError: [WinError 3] The system cannot find the path specified: '/kaggle/input/butterfly-dataset/leedsbutterfly/images'

```

In [6]: # Function to find the corresponding mask for a given image
def get_corresponding_mask(image_name):
    base_name = image_name.split('.')[0] # Extract base name (e.g., '0010001')
    mask_name = f"{base_name}_seg0.png" # Append '_seg0' to match the mask pattern
    return mask_name

```

```

In [7]: # Display a few sample images and their corresponding masks
def show_samples(image_list, num_samples=3):
    plt.figure(figsize=(10, 10))
    for i in range(num_samples):
        img = Image.open(os.path.join(image_path, image_list[i]))
        mask_name = get_corresponding_mask(image_list[i])
        mask = Image.open(os.path.join(mask_path, mask_name))

        plt.subplot(2, num_samples, i + 1)
        plt.imshow(img)
        plt.title("Image")
        plt.axis("off")

        plt.subplot(2, num_samples, num_samples + i + 1)
        plt.imshow(mask)
        plt.title("Mask")
        plt.axis("off")

    plt.show()

```

```

In [8]: # Show 3 samples
show_samples(images, num_samples=3)

```

Image



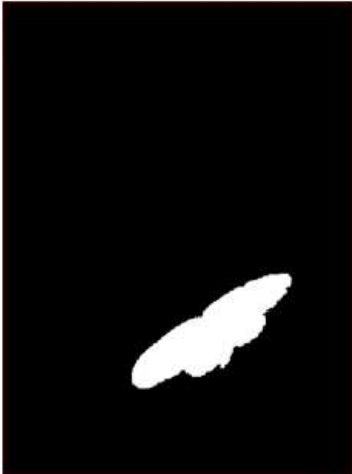
Image



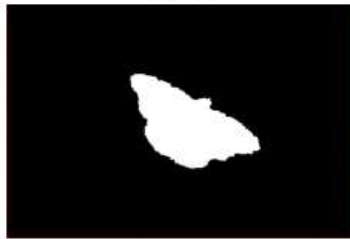
Image



Mask



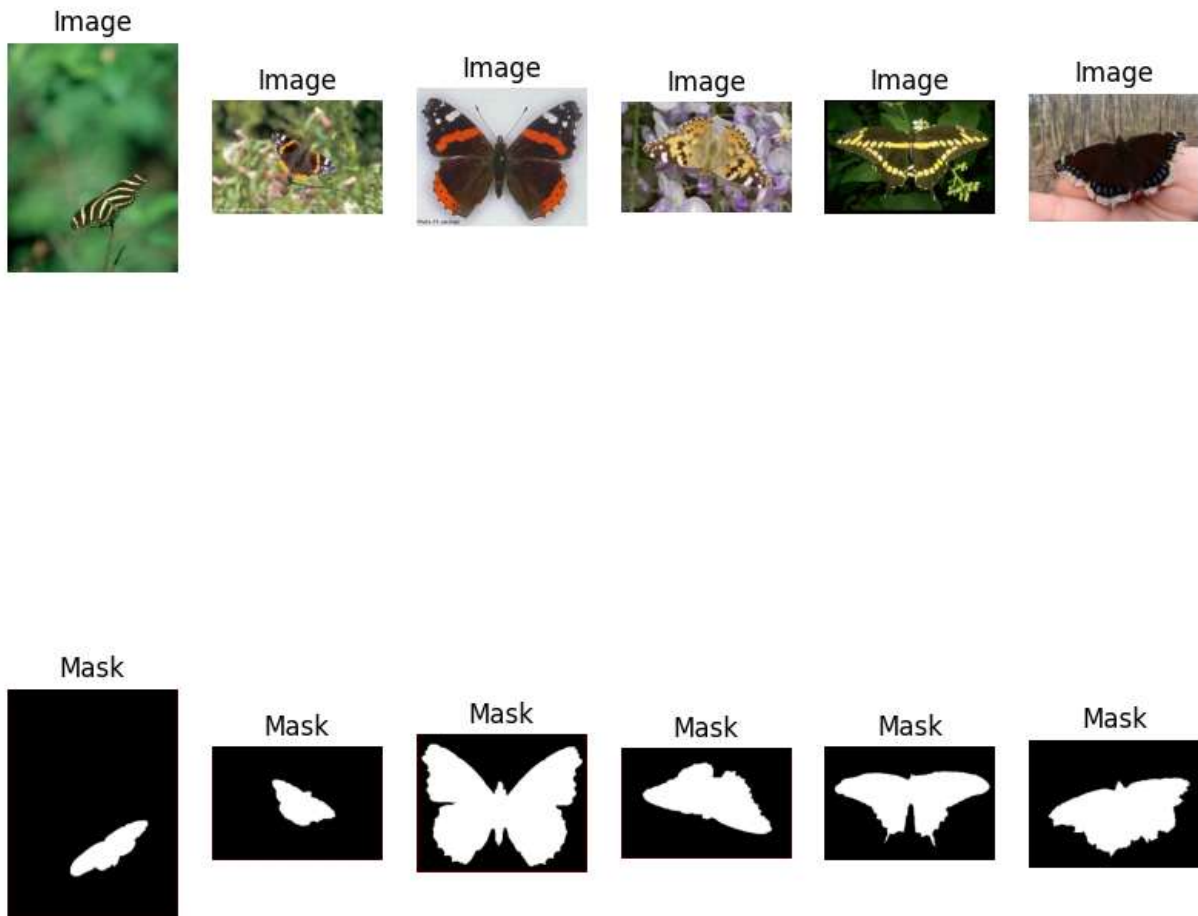
Mask



Mask



```
In [9]: # Show 3 samples  
show_samples(images, num_samples=6)
```



Load Data

- Load the data in numpy format.
- Less quality data. Total size is like 500 mb.
- If data is larger and larger, initialise data generators.
- KERAS IMAGE FROM DATASET.
- Resizing.
- INITIALISE generator for masks and then original images.

```
In [10]: # Function to Load and preprocess the dataset
def load_data(image_dir, mask_dir, image_shape=256):
    """
    Loads and preprocesses images and masks for training.

    Args:
        image_dir (str): Directory containing the images.
        mask_dir (str): Directory containing the masks.
        image_shape (int, optional): Shape to resize the images and masks. Defaults

    Returns:
        tuple: Arrays of preprocessed images and masks.
    """
    image_list = []
```



```

mask_list = []

# Create a dictionary as well, in which we have key as the original image and m

image_names = os.listdir(image_dir)
mask_names = [re.sub(r'\.png', '_seg0.png', name) for name in image_names]

for img_name, mask_name in zip(image_names, mask_names):
    try:
        img = plt.imread(os.path.join(image_dir, img_name))
        mask = plt.imread(os.path.join(mask_dir, mask_name))
    except FileNotFoundError:
        print("Exception hapened as the file is not found.")
        continue

    # Resizing the images and masks
    img_resized = cv2.resize(img, (image_shape, image_shape))
    mask_resized = cv2.resize(mask, (image_shape, image_shape))

    # Storing the preprocessed data
    image_list.append(img_resized)
    mask_list.append(mask_resized[:, :, 0]) # Use the first channel for binary

return np.array(image_list), np.array(mask_list)

```

```

In [11]: # Load data
image_dir = '/kaggle/input/butterfly-dataset/leedsbutterfly/images'
mask_dir = '/kaggle/input/butterfly-dataset/leedsbutterfly/segmentations'
images, masks = load_data(image_dir, mask_dir)

```

```

In [12]: images.shape, masks.shape

```

```

Out[12]: ((832, 256, 256, 3), (832, 256, 256))

```

Test Train Split

```

In [13]: # Split the data into training and validation sets
X_train, X_val, y_train, y_val = train_test_split(images, masks, test_size=0.2, ran

```

```

In [14]: print(X_train.shape, X_val.shape, y_train.shape, y_val.shape)

(665, 256, 256, 3) (167, 256, 256, 3) (665, 256, 256) (167, 256, 256)

```

```

In [15]: # try to do imread on these numpy arrays and then imshow on these.
X_train[0], y_train[0]

```

```

Out[15]: (array([[5.18936276e-01, 7.52047479e-01, 3.79942685e-01],
                 [5.62345684e-01, 7.80606270e-01, 4.31203276e-01],
                 [6.68411076e-01, 8.78325403e-01, 5.49136996e-01],
                 ...,
                 [1.96273118e-01, 3.57057452e-01, 5.78233600e-02],
                 [2.19239816e-01, 3.82735521e-01, 6.47974089e-02],
                 [2.28370100e-01, 4.00474876e-01, 7.71675929e-02]],

          [[5.07382810e-01, 7.45265782e-01, 3.57068777e-01],
           [5.20916462e-01, 7.47130752e-01, 3.70118678e-01],
           [5.73132992e-01, 7.91408122e-01, 4.25978333e-01],
           ...,
           [2.23232642e-01, 3.85508597e-01, 8.60773176e-02],
           [2.42049634e-01, 4.07239228e-01, 8.89016539e-02],
           [2.43650436e-01, 4.20121014e-01, 9.72196758e-02]],

          [[5.04146934e-01, 7.48984516e-01, 3.57722521e-01],
           [5.04444361e-01, 7.40914106e-01, 3.49345088e-01],
           [5.11610210e-01, 7.41482198e-01, 3.47163290e-01],
           ...,
           [2.20205277e-01, 3.87079448e-01, 8.65272731e-02],
           [2.32285962e-01, 4.02613789e-01, 8.26153010e-02],
           [2.26895049e-01, 4.11208779e-01, 9.35617238e-02]],

          ...,

          [[4.66452204e-02, 1.41666681e-01, 8.50183889e-04],
           [3.2227329e-02, 1.26340389e-01, 2.62361427e-04],
           [2.80484073e-02, 1.18244484e-01, 7.84313772e-03],
           ...,
           [1.26148900e-02, 7.26646036e-02, 0.00000000e+00],
           [1.48360915e-02, 5.63744940e-02, 7.31853396e-03],
           [1.19159473e-02, 5.59033863e-02, 2.36352673e-03]],

          [[3.57689969e-02, 1.33119270e-01, 2.62714480e-03],
           [1.91582777e-02, 1.15503624e-01, 7.98902358e-04],
           [2.40233783e-02, 1.19473748e-01, 6.84548216e-03],
           ...,
           [1.83134191e-02, 7.06878155e-02, 0.00000000e+00],
           [1.62542313e-02, 4.99628708e-02, 7.88909290e-03],
           [2.39671599e-02, 6.18501306e-02, 1.99219957e-03]],

          [[2.65471824e-02, 1.25490203e-01, 0.00000000e+00],
           [2.74509806e-02, 1.25490203e-01, 2.42034323e-03],
           [1.68811288e-02, 1.14920340e-01, 5.97426493e-04],
           ...,
           [1.96078438e-02, 6.93933889e-02, 0.00000000e+00],
           [1.32659320e-02, 4.82689962e-02, 2.71139713e-03],
           [3.61825973e-02, 7.14767203e-02, 6.54871343e-03]]], dtype=float32),
array([[0.6588211 , 0.5566406 , 0.5566406 , ..., 0.08480072, 0.5566406 ,
        0.23046875],
       [0.07607269, 0.          , 0.          , ..., 0.          , 0.          ,
        0.07607269],
       [0.23046875, 0.          , 0.          , ..., 0.          , 0.          ,
        0.23046875],
       ...,

```

```
[0.0499649 , 0.          , 0.          , ..., 0.          , 0.          ,
 0.0499649 ],
[0.23046875, 0.          , 0.          , ..., 0.          , 0.          ,
 0.23046875],
[0.6588211 , 0.5566406 , 0.5566406 , ..., 0.08480072, 0.5566406 ,
 0.23046875]], dtype=float32))
```

Let's Start Building UNET Architecture

Building Convolution Block

```
In [16]: # Function to create a convolutional block for the U-Net model
def convolutional_block(input_tensor, num_filters, kernel_size=3, use_batch_norm=True):
    """
    Performs two convolution operations with optional batch normalization.

    Args:
        input_tensor (tensor): Input to the convolutional block.
        num_filters (int): Number of filters for the convolutional layers.
        kernel_size (int, optional): Size of the convolution kernel. Defaults to 3.
        use_batch_norm (bool, optional): Apply batch normalization if True. Default

    Returns:
        tensor: Output after applying convolution, batch normalization, and activation.
    """
    # First convolution
    x = tf.keras.layers.Conv2D(filters=num_filters, kernel_size=(kernel_size, kernel_size),
                                kernel_initializer='he_normal', padding='same')(input_tensor)
    if use_batch_norm:
        x = tf.keras.layers.BatchNormalization()(x)
    x = tf.keras.layers.Activation('relu')(x)

    # Second convolution
    x = tf.keras.layers.Conv2D(filters=num_filters, kernel_size=(kernel_size, kernel_size),
                                kernel_initializer='he_normal', padding='same')(x)
    if use_batch_norm:
        x = tf.keras.layers.BatchNormalization()(x)
    x = tf.keras.layers.Activation('relu')(x)

    return x
```

Building UNET Model

```
In [17]: # Function to create a U-Net model
def build_unet(input_shape, num_filters=16, dropout_rate=0.1, use_batch_norm=True):
    """
    Builds a U-Net model.

    Args:
        input_shape (tuple): Shape of the input image (height, width, channels).
        num_filters (int, optional): Number of filters for the initial layer. Default
```

dropout_rate (float, optional): Dropout rate for regularization. Defaults to 0.5.
use_batch_norm (bool, optional): Apply batch normalization if True. Default is False.

Returns:

keras.Model: Compiled U-Net model.

"""

```
inputs = tf.keras.layers.Input(input_shape)

# Encoder path
c1 = convolutional_block(inputs, num_filters * 1, use_batch_norm=use_batch_norm)
p1 = tf.keras.layers.MaxPooling2D((2, 2))(c1)
p1 = tf.keras.layers.Dropout(dropout_rate)(p1)

c2 = convolutional_block(p1, num_filters * 2, use_batch_norm=use_batch_norm)
p2 = tf.keras.layers.MaxPooling2D((2, 2))(c2)
p2 = tf.keras.layers.Dropout(dropout_rate)(p2)

c3 = convolutional_block(p2, num_filters * 4, use_batch_norm=use_batch_norm)
p3 = tf.keras.layers.MaxPooling2D((2, 2))(c3)
p3 = tf.keras.layers.Dropout(dropout_rate)(p3)

c4 = convolutional_block(p3, num_filters * 8, use_batch_norm=use_batch_norm)
p4 = tf.keras.layers.MaxPooling2D((2, 2))(c4)
p4 = tf.keras.layers.Dropout(dropout_rate)(p4)

# Bottleneck
c5 = convolutional_block(p4, num_filters * 16, use_batch_norm=use_batch_norm)

# Decoder path
u6 = tf.keras.layers.Conv2DTranspose(num_filters * 8, (3, 3), strides=(2, 2), padding='same')(p4)
u6 = tf.keras.layers.concatenate([u6, c4])
u6 = tf.keras.layers.Dropout(dropout_rate)(u6)
c6 = convolutional_block(u6, num_filters * 8, use_batch_norm=use_batch_norm)

u7 = tf.keras.layers.Conv2DTranspose(num_filters * 4, (3, 3), strides=(2, 2), padding='same')(p3)
u7 = tf.keras.layers.concatenate([u7, c3])
u7 = tf.keras.layers.Dropout(dropout_rate)(u7)
c7 = convolutional_block(u7, num_filters * 4, use_batch_norm=use_batch_norm)

u8 = tf.keras.layers.Conv2DTranspose(num_filters * 2, (3, 3), strides=(2, 2), padding='same')(p2)
u8 = tf.keras.layers.concatenate([u8, c2])
u8 = tf.keras.layers.Dropout(dropout_rate)(u8)
c8 = convolutional_block(u8, num_filters * 2, use_batch_norm=use_batch_norm)

u9 = tf.keras.layers.Conv2DTranspose(num_filters * 1, (3, 3), strides=(2, 2), padding='same')(p1)
u9 = tf.keras.layers.concatenate([u9, c1])
u9 = tf.keras.layers.Dropout(dropout_rate)(u9)
c9 = convolutional_block(u9, num_filters * 1, use_batch_norm=use_batch_norm)

# Output layer
output = tf.keras.layers.Conv2D(1, (1, 1), activation='sigmoid')(c9)

# Model creation
model = tf.keras.Model(inputs=[inputs], outputs=[output])
return model
```

```
In [18]: # Instantiate and compile the U-Net model
input_shape = (256, 256, 3)
UNET_model = build_UNET(input_shape, dropout_rate=0.07)
UNET_model.compile(optimizer='Adam', loss='binary_crossentropy', metrics=['accuracy
```

```
I0000 00:00:1754479589.695614      36 gpu_device.cc:2022] Created device /job:localh
ost/replica:0/task:0/device:GPU:0 with 13942 MB memory:  -> device: 0, name: Tesla T
4, pci bus id: 0000:00:04.0, compute capability: 7.5
I0000 00:00:1754479589.696400      36 gpu_device.cc:2022] Created device /job:localh
ost/replica:0/task:0/device:GPU:1 with 13942 MB memory:  -> device: 1, name: Tesla T
4, pci bus id: 0000:00:05.0, compute capability: 7.5
```

```
In [19]: UNET_model.summary()
```

Model: "functional"

Layer (type)	Output Shape	Param #	Connected to
input_layer (InputLayer)	(None, 256, 256, 3)	0	-
conv2d (Conv2D)	(None, 256, 256, 16)	448	input_layer[0][0]
batch_normalization (BatchNormalizatio...	(None, 256, 256, 16)	64	conv2d[0][0]
activation (Activation)	(None, 256, 256, 16)	0	batch_normalizat...
conv2d_1 (Conv2D)	(None, 256, 256, 16)	2,320	activation[0][0]
batch_normalizatio... (BatchNormalizatio...	(None, 256, 256, 16)	64	conv2d_1[0][0]
activation_1 (Activation)	(None, 256, 256, 16)	0	batch_normalizat...
max_pooling2d (MaxPooling2D)	(None, 128, 128, 16)	0	activation_1[0][...
dropout (Dropout)	(None, 128, 128, 16)	0	max_pooling2d[0]...
conv2d_2 (Conv2D)	(None, 128, 128, 32)	4,640	dropout[0][0]
batch_normalizatio... (BatchNormalizatio...	(None, 128, 128, 32)	128	conv2d_2[0][0]
activation_2 (Activation)	(None, 128, 128, 32)	0	batch_normalizat...
conv2d_3 (Conv2D)	(None, 128, 128, 32)	9,248	activation_2[0][...
batch_normalizatio... (BatchNormalizatio...	(None, 128, 128, 32)	128	conv2d_3[0][0]
activation_3 (Activation)	(None, 128, 128, 32)	0	batch_normalizat...
max_pooling2d_1 (MaxPooling2D)	(None, 64, 64, 32)	0	activation_3[0][...
dropout_1 (Dropout)	(None, 64, 64, 32)	0	max_pooling2d_1[...
conv2d_4 (Conv2D)	(None, 64, 64, 64)	18,496	dropout_1[0][0]

batch_normalizatio... (BatchNormalizatio...	(None, 64, 64, 64)	256	conv2d_4[0][0]
activation_4 (Activation)	(None, 64, 64, 64)	0	batch_normalizat...
conv2d_5 (Conv2D)	(None, 64, 64, 64)	36,928	activation_4[0][...
batch_normalizatio... (BatchNormalizatio...	(None, 64, 64, 64)	256	conv2d_5[0][0]
activation_5 (Activation)	(None, 64, 64, 64)	0	batch_normalizat...
max_pooling2d_2 (MaxPooling2D)	(None, 32, 32, 64)	0	activation_5[0][...
dropout_2 (Dropout)	(None, 32, 32, 64)	0	max_pooling2d_2[...
conv2d_6 (Conv2D)	(None, 32, 32, 128)	73,856	dropout_2[0][0]
batch_normalizatio... (BatchNormalizatio...	(None, 32, 32, 128)	512	conv2d_6[0][0]
activation_6 (Activation)	(None, 32, 32, 128)	0	batch_normalizat...
conv2d_7 (Conv2D)	(None, 32, 32, 128)	147,584	activation_6[0][...
batch_normalizatio... (BatchNormalizatio...	(None, 32, 32, 128)	512	conv2d_7[0][0]
activation_7 (Activation)	(None, 32, 32, 128)	0	batch_normalizat...
max_pooling2d_3 (MaxPooling2D)	(None, 16, 16, 128)	0	activation_7[0][...
dropout_3 (Dropout)	(None, 16, 16, 128)	0	max_pooling2d_3[...
conv2d_8 (Conv2D)	(None, 16, 16, 256)	295,168	dropout_3[0][0]
batch_normalizatio... (BatchNormalizatio...	(None, 16, 16, 256)	1,024	conv2d_8[0][0]
activation_8 (Activation)	(None, 16, 16, 256)	0	batch_normalizat...
conv2d_9 (Conv2D)	(None, 16, 16,	590,080	activation_8[0][...

	256)		
batch_normalizatio... (BatchNormalizatio...	(None, 16, 16, 256)	1,024	conv2d_9[0][0]
activation_9 (Activation)	(None, 16, 16, 256)	0	batch_normalizat...
conv2d_transpose (Conv2DTranspose)	(None, 32, 32, 128)	295,040	activation_9[0][...
concatenate (Concatenate)	(None, 32, 32, 256)	0	conv2d_transpose... activation_7[0][...
dropout_4 (Dropout)	(None, 32, 32, 256)	0	concatenate[0][0]
conv2d_10 (Conv2D)	(None, 32, 32, 128)	295,040	dropout_4[0][0]
batch_normalizatio... (BatchNormalizatio...	(None, 32, 32, 128)	512	conv2d_10[0][0]
activation_10 (Activation)	(None, 32, 32, 128)	0	batch_normalizat...
conv2d_11 (Conv2D)	(None, 32, 32, 128)	147,584	activation_10[0]...
batch_normalizatio... (BatchNormalizatio...	(None, 32, 32, 128)	512	conv2d_11[0][0]
activation_11 (Activation)	(None, 32, 32, 128)	0	batch_normalizat...
conv2d_transpose_1 (Conv2DTranspose)	(None, 64, 64, 64)	73,792	activation_11[0]...
concatenate_1 (Concatenate)	(None, 64, 64, 128)	0	conv2d_transpose... activation_5[0][...
dropout_5 (Dropout)	(None, 64, 64, 128)	0	concatenate_1[0]...
conv2d_12 (Conv2D)	(None, 64, 64, 64)	73,792	dropout_5[0][0]
batch_normalizatio... (BatchNormalizatio...	(None, 64, 64, 64)	256	conv2d_12[0][0]
activation_12 (Activation)	(None, 64, 64, 64)	0	batch_normalizat...
conv2d_13 (Conv2D)	(None, 64, 64, 64)	36,928	activation_12[0]...

batch_normalizatio... (BatchNormalizatio...	(None, 64, 64, 64)	256	conv2d_13[0][0]
activation_13 (Activation)	(None, 64, 64, 64)	0	batch_normalizat...
conv2d_transpose_2 (Conv2DTranspose)	(None, 128, 128, 32)	18,464	activation_13[0]...
concatenate_2 (Concatenate)	(None, 128, 128, 64)	0	conv2d_transpose... activation_3[0][...
dropout_6 (Dropout)	(None, 128, 128, 64)	0	concatenate_2[0]...
conv2d_14 (Conv2D)	(None, 128, 128, 32)	18,464	dropout_6[0][0]
batch_normalizatio... (BatchNormalizatio...	(None, 128, 128, 32)	128	conv2d_14[0][0]
activation_14 (Activation)	(None, 128, 128, 32)	0	batch_normalizat...
conv2d_15 (Conv2D)	(None, 128, 128, 32)	9,248	activation_14[0]...
batch_normalizatio... (BatchNormalizatio...	(None, 128, 128, 32)	128	conv2d_15[0][0]
activation_15 (Activation)	(None, 128, 128, 32)	0	batch_normalizat...
conv2d_transpose_3 (Conv2DTranspose)	(None, 256, 256, 16)	4,624	activation_15[0]...
concatenate_3 (Concatenate)	(None, 256, 256, 32)	0	conv2d_transpose... activation_1[0][...
dropout_7 (Dropout)	(None, 256, 256, 32)	0	concatenate_3[0]...
conv2d_16 (Conv2D)	(None, 256, 256, 16)	4,624	dropout_7[0][0]
batch_normalizatio... (BatchNormalizatio...	(None, 256, 256, 16)	64	conv2d_16[0][0]
activation_16 (Activation)	(None, 256, 256, 16)	0	batch_normalizat...
conv2d_17 (Conv2D)	(None, 256, 256, 16)	2,320	activation_16[0]...
batch_normalizatio... (BatchNormalizatio...	(None, 256, 256, 16)	64	conv2d_17[0][0]

activation_17 (Activation)	(None, 256, 256, 16)	0	batch_normalizat...
conv2d_18 (Conv2D)	(None, 256, 256, 1)	17	activation_17[0]...

Total params: 2,164,593 (8.26 MB)

Trainable params: 2,161,649 (8.25 MB)

Non-trainable params: 2,944 (11.50 KB)

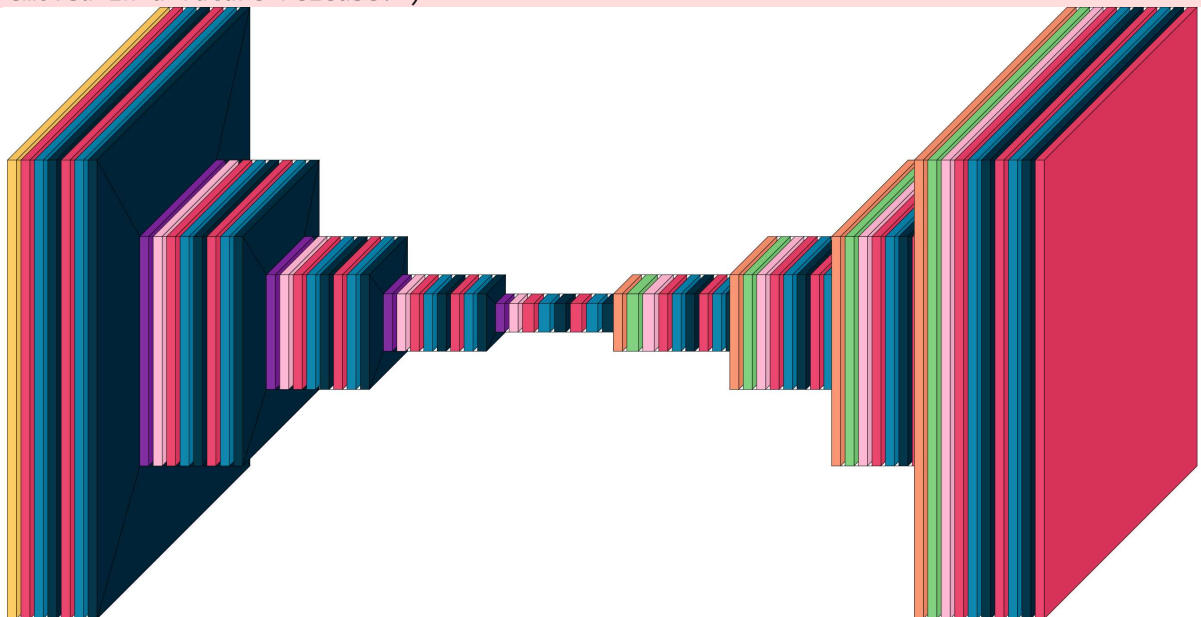
```
In [20]: %%capture
!pip install visualkeras
```

```
In [21]: from visualkeras import layered_view
import visualkeras
visualkeras.layered_view(unet_model)
```

/usr/local/lib/python3.11/dist-packages/visualkeras/layered.py:86: UserWarning: The legend_text_spacing_offset parameter is deprecated and will be removed in a future release.

warnings.warn("The legend_text_spacing_offset parameter is deprecated and will be removed in a future release.")

Out[21]:



```
In [39]: # Callbacks
early_stopping = EarlyStopping(monitor='val_loss', patience=10, verbose=1, restore_
reduce_lr = ReduceLROnPlateau(monitor='val_loss', factor=0.1, patience=5, verbose=1
model_checkpoint = ModelCheckpoint('unet_model_best.keras', monitor='val_loss', sav
```

```
In [40]: # Train the model
history = unet_model.fit(X_train,
                        y_train,
                        validation_data=(X_val, y_val),
                        epochs=50,
```

```
verbose=1,  
callbacks=[early_stopping, reduce_lr, model_checkpoint])
```

Epoch 1/50

/usr/local/lib/python3.11/dist-packages/keras/src/models/functional.py:237: UserWarning: The structure of `inputs` doesn't match the expected structure.

Expected: ['keras_tensor']

Received: inputs=Tensor(shape=(None, 256, 256, 3))

warnings.warn(msg)

WARNING: All log messages before absl::InitializeLog() is called are written to STDERR

I0000 00:00:1754481381.496142 88 service.cc:148] XLA service 0x7afae4005230 initialized for platform CUDA (this does not guarantee that XLA will be used). Devices:

I0000 00:00:1754481381.501981 88 service.cc:156] StreamExecutor device (0): Tesla T4, Compute Capability 7.5

I0000 00:00:1754481381.502006 88 service.cc:156] StreamExecutor device (1): Tesla T4, Compute Capability 7.5

I0000 00:00:1754481383.769527 88 cuda_dnn.cc:529] Loaded cuDNN version 90300

2025-08-06 11:56:49.721649: E external/local_xla/xla/service/slow_operation_alarm.cc:65] Trying algorithm eng0{} for conv (f32[32,16,3,3]{3,2,1,0}, u8[0]{0}) custom-call(f32[32,16,257,257]{3,2,1,0}, f32[32,32,128,128]{3,2,1,0}), window={size=3x3 stride=2x2}, dim_labels=bf01_oi01->bf01, custom_call_target="__cudnn\$convBackwardFilter", backend_config={"cudnn_conv_backend_config":{"activation_mode":"kNone","conv_result_scale":1,"leakyrelu_alpha":0,"side_input_scale":0},"force_earliest_schedule":false,"operation_queue_id":"0","wait_on_operation_queues":[]}] is taking a while...

2025-08-06 11:56:49.873067: E external/local_xla/xla/service/slow_operation_alarm.cc:133] The operation took 1.151552347s

Trying algorithm eng0{} for conv (f32[32,16,3,3]{3,2,1,0}, u8[0]{0}) custom-call(f32[32,16,257,257]{3,2,1,0}, f32[32,32,128,128]{3,2,1,0}), window={size=3x3 stride=2x2}, dim_labels=bf01_oi01->bf01, custom_call_target="__cudnn\$convBackwardFilter", backend_config={"cudnn_conv_backend_config":{"activation_mode":"kNone","conv_result_scale":1,"leakyrelu_alpha":0,"side_input_scale":0},"force_earliest_schedule":false,"operation_queue_id":"0","wait_on_operation_queues":[]}] is taking a while...

2025-08-06 11:56:51.898199: E external/local_xla/xla/service/slow_operation_alarm.cc:65] Trying algorithm eng0{} for conv (f32[16,32,3,3]{3,2,1,0}, u8[0]{0}) custom-call(f32[32,32,256,256]{3,2,1,0}, f32[32,16,256,256]{3,2,1,0}), window={size=3x3 pad=1_1x1_1}, dim_labels=bf01_oi01->bf01, custom_call_target="__cudnn\$convBackwardFilter", backend_config={"cudnn_conv_backend_config":{"activation_mode":"kNone","conv_result_scale":1,"leakyrelu_alpha":0,"side_input_scale":0},"force_earliest_schedule":false,"operation_queue_id":"0","wait_on_operation_queues":[]}] is taking a while...

2025-08-06 11:56:52.778767: E external/local_xla/xla/service/slow_operation_alarm.cc:133] The operation took 1.880665082s

Trying algorithm eng0{} for conv (f32[16,32,3,3]{3,2,1,0}, u8[0]{0}) custom-call(f32[32,32,256,256]{3,2,1,0}, f32[32,16,256,256]{3,2,1,0}), window={size=3x3 pad=1_1x1_1}, dim_labels=bf01_oi01->bf01, custom_call_target="__cudnn\$convBackwardFilter", backend_config={"cudnn_conv_backend_config":{"activation_mode":"kNone","conv_result_scale":1,"leakyrelu_alpha":0,"side_input_scale":0},"force_earliest_schedule":false,"operation_queue_id":"0","wait_on_operation_queues":[]}] is taking a while...

I0000 00:00:1754481423.914534 88 device_compiler.h:188] Compiled cluster using XLA! This line is logged at most once for the lifetime of the process.

21/21 ————— 0s 2s/step - accuracy: 0.7744 - loss: 0.5066
Epoch 1: val_loss improved from inf to 3.99106, saving model to unet_model_best.keras

21/21 ————— 105s 2s/step - accuracy: 0.7767 - loss: 0.5033 - val_accuracy: 0.3162 - val_loss: 3.9911 - learning_rate: 0.0010
Epoch 2/50

21/21 ————— 0s 314ms/step - accuracy: 0.8883 - loss: 0.3133
Epoch 2: val_loss improved from 3.99106 to 3.68887, saving model to unet_model_best.keras

21/21 ————— 8s 364ms/step - accuracy: 0.8885 - loss: 0.3128 - val_accuracy: 0.6053 - val_loss: 3.6889 - learning_rate: 0.0010
Epoch 3/50

21/21 ————— 0s 325ms/step - accuracy: 0.9091 - loss: 0.2595
Epoch 3: val_loss improved from 3.68887 to 2.17553, saving model to unet_model_best.keras

21/21 ————— 8s 374ms/step - accuracy: 0.9092 - loss: 0.2590 - val_accuracy: 0.6875 - val_loss: 2.1755 - learning_rate: 0.0010
Epoch 4/50

21/21 ————— 0s 331ms/step - accuracy: 0.9201 - loss: 0.2208
Epoch 4: val_loss improved from 2.17553 to 0.56020, saving model to unet_model_best.keras

21/21 ————— 8s 379ms/step - accuracy: 0.9202 - loss: 0.2205 - val_accuracy: 0.8473 - val_loss: 0.5602 - learning_rate: 0.0010
Epoch 5/50

21/21 ————— 0s 326ms/step - accuracy: 0.9252 - loss: 0.1957
Epoch 5: val_loss did not improve from 0.56020

21/21 ————— 7s 353ms/step - accuracy: 0.9253 - loss: 0.1955 - val_accuracy: 0.8643 - val_loss: 0.5720 - learning_rate: 0.0010
Epoch 6/50

21/21 ————— 0s 324ms/step - accuracy: 0.9329 - loss: 0.1719
Epoch 6: val_loss improved from 0.56020 to 0.40275, saving model to unet_model_best.keras

21/21 ————— 8s 371ms/step - accuracy: 0.9328 - loss: 0.1720 - val_accuracy: 0.8860 - val_loss: 0.4027 - learning_rate: 0.0010
Epoch 7/50

21/21 ————— 0s 321ms/step - accuracy: 0.9366 - loss: 0.1556
Epoch 7: val_loss improved from 0.40275 to 0.33743, saving model to unet_model_best.keras

21/21 ————— 8s 370ms/step - accuracy: 0.9366 - loss: 0.1556 - val_accuracy: 0.8956 - val_loss: 0.3374 - learning_rate: 0.0010
Epoch 8/50

21/21 ————— 0s 319ms/step - accuracy: 0.9369 - loss: 0.1496
Epoch 8: val_loss improved from 0.33743 to 0.23932, saving model to unet_model_best.keras

21/21 ————— 8s 366ms/step - accuracy: 0.9370 - loss: 0.1493 - val_accuracy: 0.9154 - val_loss: 0.2393 - learning_rate: 0.0010
Epoch 9/50

21/21 ————— 0s 315ms/step - accuracy: 0.9427 - loss: 0.1306
Epoch 9: val_loss improved from 0.23932 to 0.21213, saving model to unet_model_best.keras

21/21 ————— 8s 362ms/step - accuracy: 0.9427 - loss: 0.1305 - val_accuracy: 0.9205 - val_loss: 0.2121 - learning_rate: 0.0010
Epoch 10/50

21/21 ————— 0s 313ms/step - accuracy: 0.9464 - loss: 0.1185
Epoch 10: val_loss did not improve from 0.21213

21/21 ————— 7s 340ms/step - accuracy: 0.9464 - loss: 0.1185 - val_acc

uracy: 0.9072 - val_loss: 0.2703 - learning_rate: 0.0010
Epoch 11/50
21/21 ————— 0s 312ms/step - accuracy: 0.9419 - loss: 0.1251
Epoch 11: val_loss improved from 0.21213 to 0.16735, saving model to unet_model_best.keras
21/21 ————— 8s 358ms/step - accuracy: 0.9420 - loss: 0.1248 - val_accuracy: 0.9326 - val_loss: 0.1673 - learning_rate: 0.0010
Epoch 12/50
21/21 ————— 0s 311ms/step - accuracy: 0.9492 - loss: 0.1065
Epoch 12: val_loss did not improve from 0.16735
21/21 ————— 7s 337ms/step - accuracy: 0.9492 - loss: 0.1064 - val_accuracy: 0.9238 - val_loss: 0.2198 - learning_rate: 0.0010
Epoch 13/50
21/21 ————— 0s 312ms/step - accuracy: 0.9482 - loss: 0.1072
Epoch 13: val_loss improved from 0.16735 to 0.14538, saving model to unet_model_best.keras
21/21 ————— 7s 358ms/step - accuracy: 0.9481 - loss: 0.1073 - val_accuracy: 0.9378 - val_loss: 0.1454 - learning_rate: 0.0010
Epoch 14/50
21/21 ————— 0s 312ms/step - accuracy: 0.9495 - loss: 0.1005
Epoch 14: val_loss improved from 0.14538 to 0.13520, saving model to unet_model_best.keras
21/21 ————— 8s 358ms/step - accuracy: 0.9495 - loss: 0.1004 - val_accuracy: 0.9391 - val_loss: 0.1352 - learning_rate: 0.0010
Epoch 15/50
21/21 ————— 0s 314ms/step - accuracy: 0.9568 - loss: 0.0853
Epoch 15: val_loss improved from 0.13520 to 0.12700, saving model to unet_model_best.keras
21/21 ————— 8s 361ms/step - accuracy: 0.9567 - loss: 0.0853 - val_accuracy: 0.9410 - val_loss: 0.1270 - learning_rate: 0.0010
Epoch 16/50
21/21 ————— 0s 316ms/step - accuracy: 0.9581 - loss: 0.0799
Epoch 16: val_loss did not improve from 0.12700
21/21 ————— 7s 343ms/step - accuracy: 0.9580 - loss: 0.0799 - val_accuracy: 0.9406 - val_loss: 0.1297 - learning_rate: 0.0010
Epoch 17/50
21/21 ————— 0s 318ms/step - accuracy: 0.9566 - loss: 0.0803
Epoch 17: val_loss did not improve from 0.12700
21/21 ————— 7s 345ms/step - accuracy: 0.9567 - loss: 0.0802 - val_accuracy: 0.9388 - val_loss: 0.1432 - learning_rate: 0.0010
Epoch 18/50
21/21 ————— 0s 320ms/step - accuracy: 0.9593 - loss: 0.0739
Epoch 18: val_loss improved from 0.12700 to 0.09638, saving model to unet_model_best.keras
21/21 ————— 8s 366ms/step - accuracy: 0.9593 - loss: 0.0738 - val_accuracy: 0.9520 - val_loss: 0.0964 - learning_rate: 0.0010
Epoch 19/50
21/21 ————— 0s 321ms/step - accuracy: 0.9612 - loss: 0.0690
Epoch 19: val_loss did not improve from 0.09638
21/21 ————— 7s 347ms/step - accuracy: 0.9611 - loss: 0.0691 - val_accuracy: 0.9509 - val_loss: 0.0974 - learning_rate: 0.0010
Epoch 20/50
21/21 ————— 0s 320ms/step - accuracy: 0.9623 - loss: 0.0657
Epoch 20: val_loss did not improve from 0.09638
21/21 ————— 7s 347ms/step - accuracy: 0.9623 - loss: 0.0658 - val_accuracy: 0.9481 - val_loss: 0.0976 - learning_rate: 0.0010

Epoch 21/50
21/21 ————— 0s 319ms/step - accuracy: 0.9621 - loss: 0.0653
Epoch 21: val_loss improved from 0.09638 to 0.09030, saving model to unet_model_best.keras
21/21 ————— 8s 366ms/step - accuracy: 0.9621 - loss: 0.0652 - val_accuracy: 0.9528 - val_loss: 0.0903 - learning_rate: 0.0010
Epoch 22/50
21/21 ————— 0s 317ms/step - accuracy: 0.9639 - loss: 0.0610
Epoch 22: val_loss improved from 0.09030 to 0.08352, saving model to unet_model_best.keras
21/21 ————— 8s 363ms/step - accuracy: 0.9639 - loss: 0.0610 - val_accuracy: 0.9548 - val_loss: 0.0835 - learning_rate: 0.0010
Epoch 23/50
21/21 ————— 0s 316ms/step - accuracy: 0.9648 - loss: 0.0572
Epoch 23: val_loss did not improve from 0.08352
21/21 ————— 7s 343ms/step - accuracy: 0.9648 - loss: 0.0571 - val_accuracy: 0.9529 - val_loss: 0.0944 - learning_rate: 0.0010
Epoch 24/50
21/21 ————— 0s 316ms/step - accuracy: 0.9662 - loss: 0.0546
Epoch 24: val_loss improved from 0.08352 to 0.08168, saving model to unet_model_best.keras
21/21 ————— 8s 365ms/step - accuracy: 0.9661 - loss: 0.0547 - val_accuracy: 0.9571 - val_loss: 0.0817 - learning_rate: 0.0010
Epoch 25/50
21/21 ————— 0s 315ms/step - accuracy: 0.9663 - loss: 0.0529
Epoch 25: val_loss improved from 0.08168 to 0.07374, saving model to unet_model_best.keras
21/21 ————— 8s 362ms/step - accuracy: 0.9663 - loss: 0.0529 - val_accuracy: 0.9589 - val_loss: 0.0737 - learning_rate: 0.0010
Epoch 26/50
21/21 ————— 0s 313ms/step - accuracy: 0.9662 - loss: 0.0523
Epoch 26: val_loss did not improve from 0.07374
21/21 ————— 7s 339ms/step - accuracy: 0.9662 - loss: 0.0523 - val_accuracy: 0.9501 - val_loss: 0.1043 - learning_rate: 0.0010
Epoch 27/50
21/21 ————— 0s 316ms/step - accuracy: 0.9661 - loss: 0.0521
Epoch 27: val_loss did not improve from 0.07374
21/21 ————— 7s 342ms/step - accuracy: 0.9661 - loss: 0.0521 - val_accuracy: 0.9569 - val_loss: 0.0813 - learning_rate: 0.0010
Epoch 28/50
21/21 ————— 0s 315ms/step - accuracy: 0.9663 - loss: 0.0509
Epoch 28: val_loss did not improve from 0.07374
21/21 ————— 7s 341ms/step - accuracy: 0.9663 - loss: 0.0510 - val_accuracy: 0.9584 - val_loss: 0.0745 - learning_rate: 0.0010
Epoch 29/50
21/21 ————— 0s 316ms/step - accuracy: 0.9669 - loss: 0.0495
Epoch 29: val_loss improved from 0.07374 to 0.06884, saving model to unet_model_best.keras
21/21 ————— 8s 362ms/step - accuracy: 0.9669 - loss: 0.0495 - val_accuracy: 0.9610 - val_loss: 0.0688 - learning_rate: 0.0010
Epoch 30/50
21/21 ————— 0s 315ms/step - accuracy: 0.9683 - loss: 0.0459
Epoch 30: val_loss improved from 0.06884 to 0.06201, saving model to unet_model_best.keras
21/21 ————— 8s 362ms/step - accuracy: 0.9683 - loss: 0.0459 - val_accuracy: 0.9634 - val_loss: 0.0620 - learning_rate: 0.0010

Epoch 31/50
21/21 ————— 0s 316ms/step - accuracy: 0.9678 - loss: 0.0456
Epoch 31: val_loss did not improve from 0.06201
21/21 ————— 7s 342ms/step - accuracy: 0.9679 - loss: 0.0456 - val_acc
uracy: 0.9618 - val_loss: 0.0666 - learning_rate: 0.0010
Epoch 32/50
21/21 ————— 0s 316ms/step - accuracy: 0.9691 - loss: 0.0436
Epoch 32: val_loss did not improve from 0.06201
21/21 ————— 7s 342ms/step - accuracy: 0.9691 - loss: 0.0436 - val_acc
uracy: 0.9628 - val_loss: 0.0638 - learning_rate: 0.0010
Epoch 33/50
21/21 ————— 0s 317ms/step - accuracy: 0.9689 - loss: 0.0425
Epoch 33: val_loss did not improve from 0.06201
21/21 ————— 7s 343ms/step - accuracy: 0.9689 - loss: 0.0425 - val_acc
uracy: 0.9630 - val_loss: 0.0625 - learning_rate: 0.0010
Epoch 34/50
21/21 ————— 0s 316ms/step - accuracy: 0.9691 - loss: 0.0427
Epoch 34: val_loss improved from 0.06201 to 0.06111, saving model to unet_model_bes
t.keras
21/21 ————— 8s 362ms/step - accuracy: 0.9690 - loss: 0.0427 - val_acc
uracy: 0.9637 - val_loss: 0.0611 - learning_rate: 0.0010
Epoch 35/50
21/21 ————— 0s 316ms/step - accuracy: 0.9696 - loss: 0.0421
Epoch 35: val_loss did not improve from 0.06111
21/21 ————— 7s 342ms/step - accuracy: 0.9696 - loss: 0.0421 - val_acc
uracy: 0.9628 - val_loss: 0.0658 - learning_rate: 0.0010
Epoch 36/50
21/21 ————— 0s 316ms/step - accuracy: 0.9694 - loss: 0.0418
Epoch 36: val_loss did not improve from 0.06111
21/21 ————— 7s 342ms/step - accuracy: 0.9694 - loss: 0.0419 - val_acc
uracy: 0.9617 - val_loss: 0.0686 - learning_rate: 0.0010
Epoch 37/50
21/21 ————— 0s 317ms/step - accuracy: 0.9676 - loss: 0.0464
Epoch 37: val_loss did not improve from 0.06111
21/21 ————— 7s 345ms/step - accuracy: 0.9676 - loss: 0.0464 - val_acc
uracy: 0.9560 - val_loss: 0.0887 - learning_rate: 0.0010
Epoch 38/50
21/21 ————— 0s 319ms/step - accuracy: 0.9678 - loss: 0.0459
Epoch 38: val_loss did not improve from 0.06111
21/21 ————— 7s 345ms/step - accuracy: 0.9678 - loss: 0.0459 - val_acc
uracy: 0.9593 - val_loss: 0.0797 - learning_rate: 0.0010
Epoch 39/50
21/21 ————— 0s 318ms/step - accuracy: 0.9679 - loss: 0.0444
Epoch 39: ReduceLROnPlateau reducing learning rate to 0.00010000000474974513.

Epoch 39: val_loss did not improve from 0.06111
21/21 ————— 7s 344ms/step - accuracy: 0.9679 - loss: 0.0444 - val_acc
uracy: 0.9616 - val_loss: 0.0692 - learning_rate: 0.0010
Epoch 40/50
21/21 ————— 0s 319ms/step - accuracy: 0.9699 - loss: 0.0402
Epoch 40: val_loss did not improve from 0.06111
21/21 ————— 7s 346ms/step - accuracy: 0.9699 - loss: 0.0402 - val_acc
uracy: 0.9637 - val_loss: 0.0620 - learning_rate: 1.0000e-04
Epoch 41/50
21/21 ————— 0s 318ms/step - accuracy: 0.9699 - loss: 0.0396
Epoch 41: val_loss improved from 0.06111 to 0.05986, saving model to unet_model_bes

```

t.keras
21/21 ----- 8s 365ms/step - accuracy: 0.9699 - loss: 0.0396 - val_acc
uracy: 0.9643 - val_loss: 0.0599 - learning_rate: 1.0000e-04
Epoch 42/50
21/21 ----- 0s 318ms/step - accuracy: 0.9707 - loss: 0.0379
Epoch 42: val_loss improved from 0.05986 to 0.05899, saving model to unet_model_bes
t.keras
21/21 ----- 8s 366ms/step - accuracy: 0.9707 - loss: 0.0379 - val_acc
uracy: 0.9645 - val_loss: 0.0590 - learning_rate: 1.0000e-04
Epoch 43/50
21/21 ----- 0s 317ms/step - accuracy: 0.9705 - loss: 0.0381
Epoch 43: val_loss improved from 0.05899 to 0.05883, saving model to unet_model_bes
t.keras
21/21 ----- 8s 364ms/step - accuracy: 0.9705 - loss: 0.0381 - val_acc
uracy: 0.9646 - val_loss: 0.0588 - learning_rate: 1.0000e-04
Epoch 44/50
21/21 ----- 0s 318ms/step - accuracy: 0.9706 - loss: 0.0377
Epoch 44: val_loss did not improve from 0.05883
21/21 ----- 7s 344ms/step - accuracy: 0.9706 - loss: 0.0377 - val_acc
uracy: 0.9646 - val_loss: 0.0589 - learning_rate: 1.0000e-04
Epoch 45/50
21/21 ----- 0s 317ms/step - accuracy: 0.9709 - loss: 0.0372
Epoch 45: val_loss did not improve from 0.05883
21/21 ----- 7s 343ms/step - accuracy: 0.9709 - loss: 0.0372 - val_acc
uracy: 0.9644 - val_loss: 0.0592 - learning_rate: 1.0000e-04
Epoch 46/50
21/21 ----- 0s 318ms/step - accuracy: 0.9710 - loss: 0.0369
Epoch 46: val_loss did not improve from 0.05883
21/21 ----- 7s 345ms/step - accuracy: 0.9710 - loss: 0.0370 - val_acc
uracy: 0.9645 - val_loss: 0.0591 - learning_rate: 1.0000e-04
Epoch 47/50
21/21 ----- 0s 319ms/step - accuracy: 0.9705 - loss: 0.0377
Epoch 47: val_loss did not improve from 0.05883
21/21 ----- 7s 346ms/step - accuracy: 0.9706 - loss: 0.0377 - val_acc
uracy: 0.9645 - val_loss: 0.0590 - learning_rate: 1.0000e-04
Epoch 48/50
21/21 ----- 0s 319ms/step - accuracy: 0.9709 - loss: 0.0373
Epoch 48: ReduceLROnPlateau reducing learning rate to 1.0000000474974514e-05.

Epoch 48: val_loss did not improve from 0.05883
21/21 ----- 7s 346ms/step - accuracy: 0.9709 - loss: 0.0373 - val_acc
uracy: 0.9646 - val_loss: 0.0591 - learning_rate: 1.0000e-04
Epoch 49/50
21/21 ----- 0s 318ms/step - accuracy: 0.9711 - loss: 0.0367
Epoch 49: val_loss did not improve from 0.05883
21/21 ----- 7s 344ms/step - accuracy: 0.9711 - loss: 0.0367 - val_acc
uracy: 0.9646 - val_loss: 0.0590 - learning_rate: 1.0000e-05
Epoch 50/50
21/21 ----- 0s 317ms/step - accuracy: 0.9708 - loss: 0.0373
Epoch 50: val_loss did not improve from 0.05883
21/21 ----- 7s 344ms/step - accuracy: 0.9708 - loss: 0.0373 - val_acc
uracy: 0.9646 - val_loss: 0.0590 - learning_rate: 1.0000e-05
Restoring model weights from the end of the best epoch: 43.

```

```

In [41]: # Plot training and validation loss and accuracy
plt.figure(figsize=(12, 5))

```



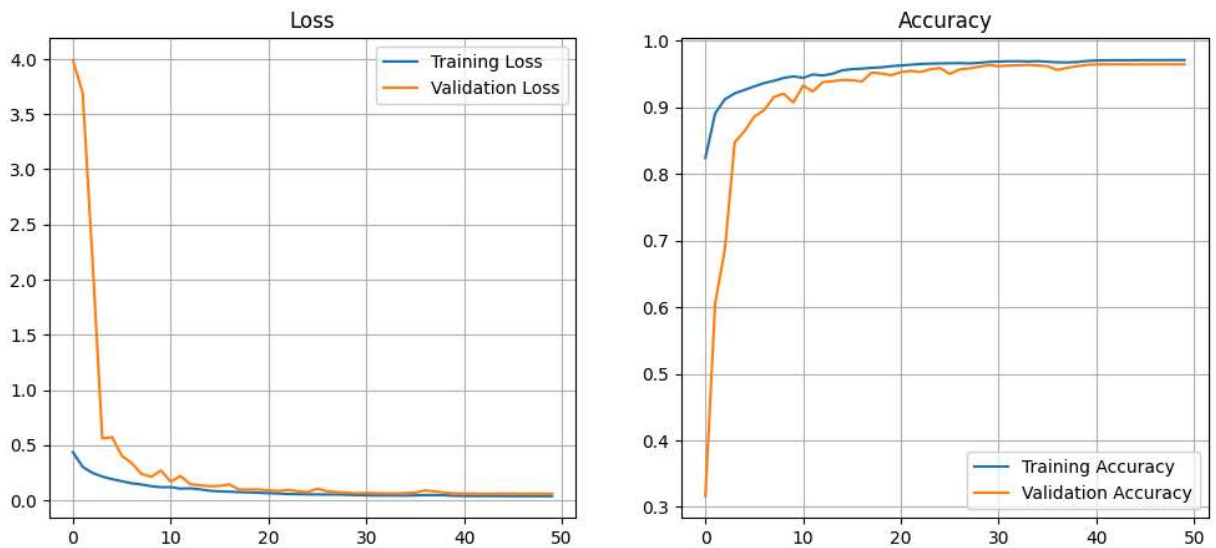
```

plt.subplot(1, 2, 1)
plt.plot(history.history['loss'], label='Training Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')
plt.legend()
plt.grid()
plt.title('Loss')

plt.subplot(1, 2, 2)
plt.plot(history.history['accuracy'], label='Training Accuracy')
plt.plot(history.history['val_accuracy'], label='Validation Accuracy')
plt.legend()
plt.grid()
plt.title('Accuracy')

plt.show()

```



```

In [42]: # Function to predict and time the inference for a single image
def predict_single_image(image, model):
    """
    Predicts the segmentation mask for a single image and times the prediction.

    Args:
        image (array): Input image for prediction.
        model (keras.Model): Trained U-Net model.

    Returns:
        tuple: Predicted mask and inference time.
    """
    start_time = time.time()
    prediction = model.predict(np.expand_dims(image, axis=0))[0, :, :, 0]
    end_time = time.time()
    inference_time = end_time - start_time

    return prediction, inference_time

```

```

In [43]: # Function to visualize the prediction and ground truth
def visualize_segmentation(image, predicted_mask, ground_truth_mask):
    """

```

Visualizes the original image, predicted mask, and ground truth mask.

Args:

image (array): Original image.
predicted_mask (array): Predicted mask.
ground_truth_mask (array): Ground truth mask.

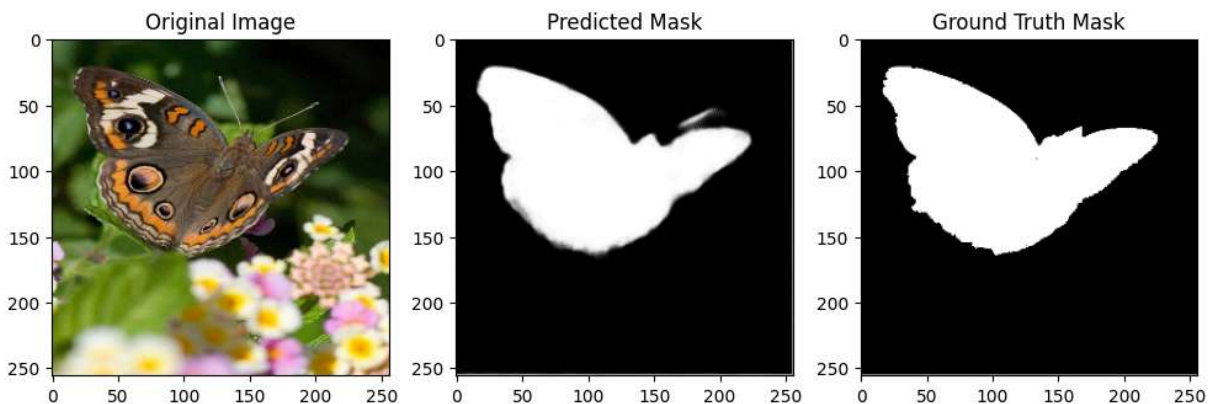
```
"""  
plt.figure(figsize=(12, 4))  
  
plt.subplot(1, 3, 1)  
plt.imshow(image)  
plt.title('Original Image')  
  
plt.subplot(1, 3, 2)  
plt.imshow(predicted_mask, cmap='gray')  
plt.title('Predicted Mask')  
  
plt.subplot(1, 3, 3)  
plt.imshow(ground_truth_mask, cmap='gray')  
plt.title('Ground Truth Mask')  
  
plt.show()
```

```
In [44]: # Example inference on a validation image  
sample_image = X_val[0]  
sample_ground_truth = y_val[0]  
  
predicted_mask, inference_time = predict_single_image(sample_image, unet_model)  
print(f'Inference time for a single image: {inference_time:.4f} seconds')  
  
# Visualize the result  
visualize_segmentation(sample_image, predicted_mask, sample_ground_truth)
```

/usr/local/lib/python3.11/dist-packages/keras/src/models/functional.py:237: UserWarning: The structure of `inputs` doesn't match the expected structure.
Expected: ['keras_tensor']
Received: inputs=Tensor(shape=(1, 256, 256, 3))
warnings.warn(msg)

1/1 ————— 3s 3s/step

Inference time for a single image: 3.4984 seconds



```
In [45]: # Example inference on a validation image  
sample_image = X_val[10]
```

```

sample_ground_truth = y_val[10]

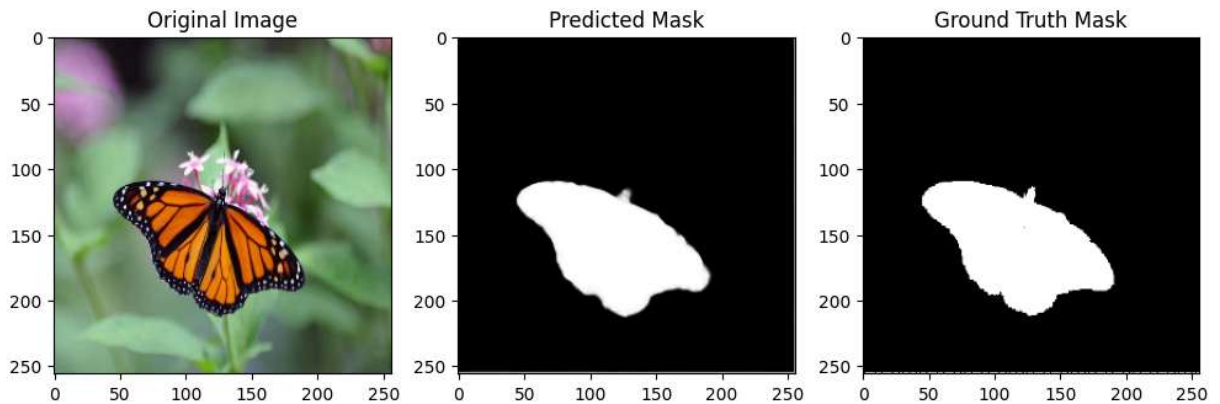
predicted_mask, inference_time = predict_single_image(sample_image, unet_model)
print(f'Inference time for a single image: {inference_time:.4f} seconds')

# Visualize the result
visualize_segmentation(sample_image, predicted_mask, sample_ground_truth)

```

1/1 ————— 0s 38ms/step

Inference time for a single image: 0.1015 seconds



Calculating Mean IOU Score

```

In [46]: import tensorflow.keras.backend as K
def mean_iou(y_true, y_pred, smooth=1):
    """
    Calculate the mean Intersection over Union (IoU).

    Args:
        y_true (tensor): Ground truth mask.
        y_pred (tensor): Predicted mask.
        smooth (float, optional): Smoothing factor to avoid division by zero. Default: 1.

    Returns:
        float: Mean IoU score.
    """
    y_true_f = K.flatten(y_true)
    y_pred_f = K.flatten(y_pred)
    intersection = K.sum(y_true_f * y_pred_f)
    union = K.sum(y_true_f) + K.sum(y_pred_f) - intersection
    return (intersection + smooth) / (union + smooth)

```

```

In [47]: # Get predictions
y_pred = unet_model.predict(X_val)
print(mean_iou(y_val, y_pred))

```

6/6 ————— 3s 230ms/step
tf.Tensor(0.88557017, shape=(), dtype=float32)

```

In [48]: # Get predictions
y_pred = unet_model.predict(X_train)
print(mean_iou(y_train, y_pred))

```

Adding IOU to the metrics while training Model

```
In [49]: import tensorflow.keras.backend as K

def dice_coefficient(y_true, y_pred, smooth=1):
    """
    Calculate the Dice coefficient.

    Args:
        y_true (tensor): Ground truth mask.
        y_pred (tensor): Predicted mask.
        smooth (float, optional): Smoothing factor to avoid division by zero. Default is 1.

    Returns:
        float: Dice coefficient score.
    """
    y_true_f = K.flatten(y_true)
    y_pred_f = K.flatten(y_pred)
    intersection = K.sum(y_true_f * y_pred_f)
    return (2. * intersection + smooth) / (K.sum(y_true_f) + K.sum(y_pred_f) + smooth)

def mean_iou(y_true, y_pred, smooth=1):
    """
    Calculate the mean Intersection over Union (IoU).

    Args:
        y_true (tensor): Ground truth mask.
        y_pred (tensor): Predicted mask.
        smooth (float, optional): Smoothing factor to avoid division by zero. Default is 1.

    Returns:
        float: Mean IoU score.
    """
    y_true_f = K.flatten(y_true)
    y_pred_f = K.flatten(y_pred)
    intersection = K.sum(y_true_f * y_pred_f)
    union = K.sum(y_true_f) + K.sum(y_pred_f) - intersection
    return (intersection + smooth) / (union + smooth)

In [50]: input_shape = (256, 256, 3)
         unet_model = build_unet(input_shape, dropout_rate=0.07)

In [51]: # Compile the U-Net model with Dice coefficient and mean IoU as metrics
         unet_model.compile(optimizer='Adam', loss='binary_crossentropy',
                           metrics=['accuracy', dice_coefficient, mean_iou])

In [52]: # Train the model with the additional metrics
         history = unet_model.fit(X_train, y_train, validation_data=(X_val, y_val), epochs=5)
```

Epoch 1/50

```
/usr/local/lib/python3.11/dist-packages/keras/src/models/functional.py:237: UserWarning: The structure of `inputs` doesn't match the expected structure.  
Expected: ['keras_tensor_76']  
Received: inputs=Tensor(shape=(None, 256, 256, 3))  
warnings.warn(msg)
```

21/21 ————— 57s 1s/step - accuracy: 0.4162 - dice_coefficient: 0.4050
- loss: 0.7810 - mean_iou: 0.2546 - val_accuracy: 0.3080 - val_dice_coefficient: 0.4191 - val_loss: 10.4565 - val_mean_iou: 0.2654
Epoch 2/50

21/21 ————— 7s 355ms/step - accuracy: 0.8548 - dice_coefficient: 0.5096 - loss: 0.5081 - mean_iou: 0.3422 - val_accuracy: 0.2920 - val_dice_coefficient: 0.4168 - val_loss: 10.2239 - val_mean_iou: 0.2635
Epoch 3/50

21/21 ————— 8s 367ms/step - accuracy: 0.8825 - dice_coefficient: 0.5510 - loss: 0.4433 - mean_iou: 0.3805 - val_accuracy: 0.3027 - val_dice_coefficient: 0.4196 - val_loss: 9.5241 - val_mean_iou: 0.2658
Epoch 4/50

21/21 ————— 8s 373ms/step - accuracy: 0.8987 - dice_coefficient: 0.5755 - loss: 0.3960 - mean_iou: 0.4044 - val_accuracy: 0.5695 - val_dice_coefficient: 0.4966 - val_loss: 3.3866 - val_mean_iou: 0.3308
Epoch 5/50

21/21 ————— 8s 363ms/step - accuracy: 0.9156 - dice_coefficient: 0.6035 - loss: 0.3558 - mean_iou: 0.4324 - val_accuracy: 0.6595 - val_dice_coefficient: 0.5354 - val_loss: 2.3213 - val_mean_iou: 0.3660
Epoch 6/50

21/21 ————— 7s 353ms/step - accuracy: 0.9085 - dice_coefficient: 0.6173 - loss: 0.3399 - mean_iou: 0.4468 - val_accuracy: 0.8620 - val_dice_coefficient: 0.6569 - val_loss: 0.5166 - val_mean_iou: 0.4895
Epoch 7/50

21/21 ————— 7s 348ms/step - accuracy: 0.9278 - dice_coefficient: 0.6499 - loss: 0.2993 - mean_iou: 0.4818 - val_accuracy: 0.8727 - val_dice_coefficient: 0.6404 - val_loss: 0.3591 - val_mean_iou: 0.4719
Epoch 8/50

21/21 ————— 7s 342ms/step - accuracy: 0.9362 - dice_coefficient: 0.6723 - loss: 0.2713 - mean_iou: 0.5067 - val_accuracy: 0.7920 - val_dice_coefficient: 0.3851 - val_loss: 0.5215 - val_mean_iou: 0.2390
Epoch 9/50

21/21 ————— 7s 339ms/step - accuracy: 0.9353 - dice_coefficient: 0.6855 - loss: 0.2560 - mean_iou: 0.5217 - val_accuracy: 0.7821 - val_dice_coefficient: 0.3612 - val_loss: 0.5519 - val_mean_iou: 0.2208
Epoch 10/50

21/21 ————— 7s 336ms/step - accuracy: 0.9407 - dice_coefficient: 0.7069 - loss: 0.2334 - mean_iou: 0.5471 - val_accuracy: 0.8803 - val_dice_coefficient: 0.6706 - val_loss: 0.3042 - val_mean_iou: 0.5068
Epoch 11/50

21/21 ————— 7s 336ms/step - accuracy: 0.9364 - dice_coefficient: 0.7212 - loss: 0.2212 - mean_iou: 0.5642 - val_accuracy: 0.8969 - val_dice_coefficient: 0.7041 - val_loss: 0.2502 - val_mean_iou: 0.5467
Epoch 12/50

21/21 ————— 7s 337ms/step - accuracy: 0.9434 - dice_coefficient: 0.7366 - loss: 0.2027 - mean_iou: 0.5832 - val_accuracy: 0.8409 - val_dice_coefficient: 0.5613 - val_loss: 0.3762 - val_mean_iou: 0.3911
Epoch 13/50

21/21 ————— 7s 339ms/step - accuracy: 0.9461 - dice_coefficient: 0.7585 - loss: 0.1857 - mean_iou: 0.6112 - val_accuracy: 0.8693 - val_dice_coefficient: 0.6257 - val_loss: 0.3100 - val_mean_iou: 0.4586
Epoch 14/50

21/21 ————— 7s 341ms/step - accuracy: 0.9490 - dice_coefficient: 0.7696 - loss: 0.1724 - mean_iou: 0.6256 - val_accuracy: 0.8879 - val_dice_coefficient: 0.6988 - val_loss: 0.2560 - val_mean_iou: 0.5376
Epoch 15/50

21/21 ————— 7s 344ms/step - accuracy: 0.9494 - dice_coefficient: 0.78
74 - loss: 0.1606 - mean_iou: 0.6496 - val_accuracy: 0.9174 - val_dice_coefficient:
0.7591 - val_loss: 0.1848 - val_mean_iou: 0.6120
Epoch 16/50

21/21 ————— 7s 346ms/step - accuracy: 0.9531 - dice_coefficient: 0.79
36 - loss: 0.1496 - mean_iou: 0.6580 - val_accuracy: 0.9021 - val_dice_coefficient:
0.7273 - val_loss: 0.2088 - val_mean_iou: 0.5719
Epoch 17/50

21/21 ————— 7s 346ms/step - accuracy: 0.9530 - dice_coefficient: 0.80
69 - loss: 0.1404 - mean_iou: 0.6764 - val_accuracy: 0.9227 - val_dice_coefficient:
0.7762 - val_loss: 0.1726 - val_mean_iou: 0.6350
Epoch 18/50

21/21 ————— 7s 348ms/step - accuracy: 0.9552 - dice_coefficient: 0.82
53 - loss: 0.1305 - mean_iou: 0.7027 - val_accuracy: 0.9018 - val_dice_coefficient:
0.7356 - val_loss: 0.2243 - val_mean_iou: 0.5823
Epoch 19/50

21/21 ————— 7s 349ms/step - accuracy: 0.9573 - dice_coefficient: 0.82
99 - loss: 0.1223 - mean_iou: 0.7094 - val_accuracy: 0.9166 - val_dice_coefficient:
0.7538 - val_loss: 0.1834 - val_mean_iou: 0.6052
Epoch 20/50

21/21 ————— 7s 347ms/step - accuracy: 0.9585 - dice_coefficient: 0.84
37 - loss: 0.1133 - mean_iou: 0.7301 - val_accuracy: 0.9398 - val_dice_coefficient:
0.8180 - val_loss: 0.1327 - val_mean_iou: 0.6923
Epoch 21/50

21/21 ————— 7s 346ms/step - accuracy: 0.9611 - dice_coefficient: 0.84
92 - loss: 0.1055 - mean_iou: 0.7380 - val_accuracy: 0.8980 - val_dice_coefficient:
0.7027 - val_loss: 0.2268 - val_mean_iou: 0.5420
Epoch 22/50

21/21 ————— 7s 345ms/step - accuracy: 0.9602 - dice_coefficient: 0.86
17 - loss: 0.1013 - mean_iou: 0.7572 - val_accuracy: 0.9455 - val_dice_coefficient:
0.8248 - val_loss: 0.1248 - val_mean_iou: 0.7019
Epoch 23/50

21/21 ————— 7s 345ms/step - accuracy: 0.9617 - dice_coefficient: 0.86
48 - loss: 0.0960 - mean_iou: 0.7619 - val_accuracy: 0.9284 - val_dice_coefficient:
0.7845 - val_loss: 0.1647 - val_mean_iou: 0.6456
Epoch 24/50

21/21 ————— 7s 343ms/step - accuracy: 0.9642 - dice_coefficient: 0.86
73 - loss: 0.0907 - mean_iou: 0.7659 - val_accuracy: 0.9434 - val_dice_coefficient:
0.8280 - val_loss: 0.1272 - val_mean_iou: 0.7067
Epoch 25/50

21/21 ————— 7s 344ms/step - accuracy: 0.9622 - dice_coefficient: 0.88
00 - loss: 0.0882 - mean_iou: 0.7860 - val_accuracy: 0.9496 - val_dice_coefficient:
0.8609 - val_loss: 0.1070 - val_mean_iou: 0.7560
Epoch 26/50

21/21 ————— 7s 345ms/step - accuracy: 0.9591 - dice_coefficient: 0.87
15 - loss: 0.0944 - mean_iou: 0.7724 - val_accuracy: 0.9188 - val_dice_coefficient:
0.8254 - val_loss: 0.1998 - val_mean_iou: 0.7030
Epoch 27/50

21/21 ————— 7s 343ms/step - accuracy: 0.9609 - dice_coefficient: 0.88
33 - loss: 0.0875 - mean_iou: 0.7911 - val_accuracy: 0.9545 - val_dice_coefficient:
0.8770 - val_loss: 0.0974 - val_mean_iou: 0.7810
Epoch 28/50

21/21 ————— 7s 344ms/step - accuracy: 0.9633 - dice_coefficient: 0.88
98 - loss: 0.0803 - mean_iou: 0.8017 - val_accuracy: 0.9544 - val_dice_coefficient:
0.8803 - val_loss: 0.0990 - val_mean_iou: 0.7862
Epoch 29/50

21/21 ————— 7s 345ms/step - accuracy: 0.9638 - dice_coefficient: 0.89
28 - loss: 0.0775 - mean_iou: 0.8065 - val_accuracy: 0.9566 - val_dice_coefficient:
0.8857 - val_loss: 0.0911 - val_mean_iou: 0.7948
Epoch 30/50

21/21 ————— 7s 344ms/step - accuracy: 0.9646 - dice_coefficient: 0.89
64 - loss: 0.0747 - mean_iou: 0.8123 - val_accuracy: 0.9549 - val_dice_coefficient:
0.8779 - val_loss: 0.0951 - val_mean_iou: 0.7825
Epoch 31/50

21/21 ————— 7s 345ms/step - accuracy: 0.9643 - dice_coefficient: 0.89
93 - loss: 0.0733 - mean_iou: 0.8170 - val_accuracy: 0.9564 - val_dice_coefficient:
0.8882 - val_loss: 0.0909 - val_mean_iou: 0.7989
Epoch 32/50

21/21 ————— 7s 345ms/step - accuracy: 0.9662 - dice_coefficient: 0.90
21 - loss: 0.0689 - mean_iou: 0.8217 - val_accuracy: 0.9587 - val_dice_coefficient:
0.8845 - val_loss: 0.0852 - val_mean_iou: 0.7930
Epoch 33/50

21/21 ————— 7s 346ms/step - accuracy: 0.9655 - dice_coefficient: 0.90
92 - loss: 0.0672 - mean_iou: 0.8338 - val_accuracy: 0.9598 - val_dice_coefficient:
0.8970 - val_loss: 0.0816 - val_mean_iou: 0.8132
Epoch 34/50

21/21 ————— 7s 347ms/step - accuracy: 0.9671 - dice_coefficient: 0.91
83 - loss: 0.0611 - mean_iou: 0.8491 - val_accuracy: 0.9581 - val_dice_coefficient:
0.8912 - val_loss: 0.0858 - val_mean_iou: 0.8038
Epoch 35/50

21/21 ————— 7s 347ms/step - accuracy: 0.9652 - dice_coefficient: 0.91
16 - loss: 0.0658 - mean_iou: 0.8378 - val_accuracy: 0.9538 - val_dice_coefficient:
0.8975 - val_loss: 0.1035 - val_mean_iou: 0.8141
Epoch 36/50

21/21 ————— 7s 347ms/step - accuracy: 0.9659 - dice_coefficient: 0.91
54 - loss: 0.0630 - mean_iou: 0.8440 - val_accuracy: 0.9584 - val_dice_coefficient:
0.9037 - val_loss: 0.0823 - val_mean_iou: 0.8244
Epoch 37/50

21/21 ————— 7s 347ms/step - accuracy: 0.9670 - dice_coefficient: 0.91
81 - loss: 0.0601 - mean_iou: 0.8486 - val_accuracy: 0.9597 - val_dice_coefficient:
0.9023 - val_loss: 0.0778 - val_mean_iou: 0.8221
Epoch 38/50

21/21 ————— 7s 347ms/step - accuracy: 0.9679 - dice_coefficient: 0.92
53 - loss: 0.0557 - mean_iou: 0.8611 - val_accuracy: 0.9623 - val_dice_coefficient:
0.9068 - val_loss: 0.0722 - val_mean_iou: 0.8295
Epoch 39/50

21/21 ————— 7s 347ms/step - accuracy: 0.9693 - dice_coefficient: 0.92
80 - loss: 0.0528 - mean_iou: 0.8656 - val_accuracy: 0.9626 - val_dice_coefficient:
0.9065 - val_loss: 0.0707 - val_mean_iou: 0.8290
Epoch 40/50

21/21 ————— 7s 348ms/step - accuracy: 0.9692 - dice_coefficient: 0.93
13 - loss: 0.0511 - mean_iou: 0.8715 - val_accuracy: 0.9639 - val_dice_coefficient:
0.9160 - val_loss: 0.0670 - val_mean_iou: 0.8451
Epoch 41/50

21/21 ————— 7s 347ms/step - accuracy: 0.9686 - dice_coefficient: 0.93
36 - loss: 0.0509 - mean_iou: 0.8756 - val_accuracy: 0.9632 - val_dice_coefficient:
0.9154 - val_loss: 0.0680 - val_mean_iou: 0.8440
Epoch 42/50

21/21 ————— 7s 346ms/step - accuracy: 0.9699 - dice_coefficient: 0.93
48 - loss: 0.0484 - mean_iou: 0.8776 - val_accuracy: 0.9640 - val_dice_coefficient:
0.9165 - val_loss: 0.0653 - val_mean_iou: 0.8459
Epoch 43/50

21/21 ————— 7s 346ms/step - accuracy: 0.9700 - dice_coefficient: 0.9358 - loss: 0.0479 - mean_iou: 0.8794 - val_accuracy: 0.9637 - val_dice_coefficient: 0.9187 - val_loss: 0.0660 - val_mean_iou: 0.8497
Epoch 44/50

21/21 ————— 7s 346ms/step - accuracy: 0.9702 - dice_coefficient: 0.9373 - loss: 0.0467 - mean_iou: 0.8821 - val_accuracy: 0.9645 - val_dice_coefficient: 0.9223 - val_loss: 0.0638 - val_mean_iou: 0.8558
Epoch 45/50

21/21 ————— 7s 347ms/step - accuracy: 0.9706 - dice_coefficient: 0.9384 - loss: 0.0457 - mean_iou: 0.8840 - val_accuracy: 0.9643 - val_dice_coefficient: 0.9214 - val_loss: 0.0636 - val_mean_iou: 0.8543
Epoch 46/50

21/21 ————— 7s 347ms/step - accuracy: 0.9701 - dice_coefficient: 0.9399 - loss: 0.0454 - mean_iou: 0.8866 - val_accuracy: 0.9639 - val_dice_coefficient: 0.9237 - val_loss: 0.0638 - val_mean_iou: 0.8582
Epoch 47/50

21/21 ————— 7s 347ms/step - accuracy: 0.9709 - dice_coefficient: 0.9412 - loss: 0.0439 - mean_iou: 0.8890 - val_accuracy: 0.9649 - val_dice_coefficient: 0.9234 - val_loss: 0.0614 - val_mean_iou: 0.8577
Epoch 48/50

21/21 ————— 7s 347ms/step - accuracy: 0.9708 - dice_coefficient: 0.9423 - loss: 0.0434 - mean_iou: 0.8909 - val_accuracy: 0.9646 - val_dice_coefficient: 0.9255 - val_loss: 0.0620 - val_mean_iou: 0.8614
Epoch 49/50

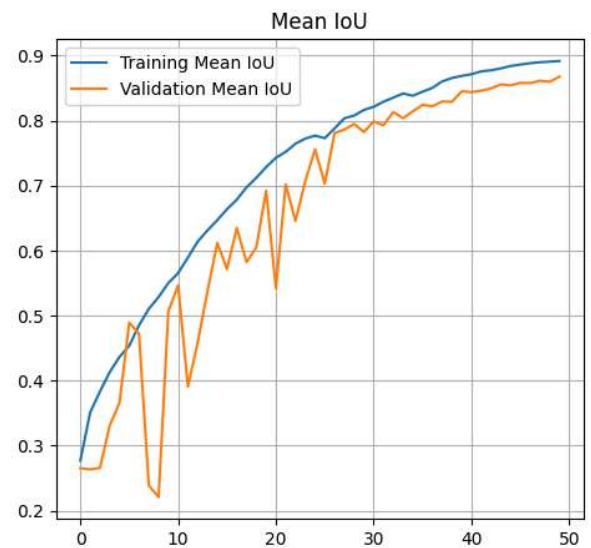
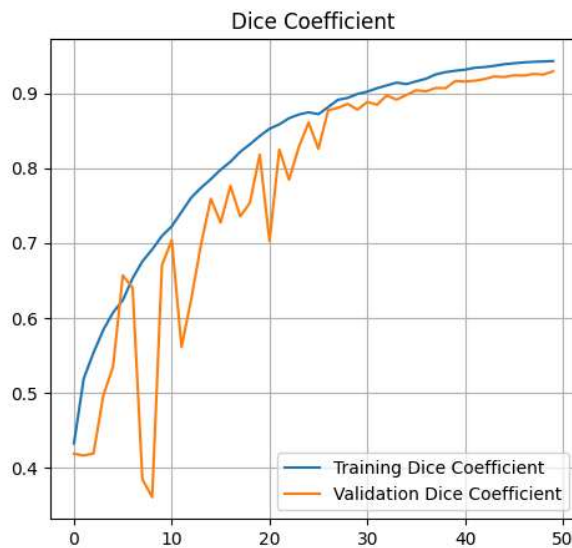
21/21 ————— 7s 347ms/step - accuracy: 0.9701 - dice_coefficient: 0.9427 - loss: 0.0439 - mean_iou: 0.8917 - val_accuracy: 0.9626 - val_dice_coefficient: 0.9246 - val_loss: 0.0675 - val_mean_iou: 0.8599
Epoch 50/50

21/21 ————— 7s 345ms/step - accuracy: 0.9709 - dice_coefficient: 0.9413 - loss: 0.0433 - mean_iou: 0.8892 - val_accuracy: 0.9643 - val_dice_coefficient: 0.9291 - val_loss: 0.0622 - val_mean_iou: 0.8676

```
In [53]: # Plot Dice coefficient and mean IoU during training and validation
plt.figure(figsize=(12, 5))
plt.subplot(1, 2, 1)
plt.plot(history.history['dice_coefficient'], label='Training Dice Coefficient')
plt.plot(history.history['val_dice_coefficient'], label='Validation Dice Coefficient')
plt.legend()
plt.grid()
plt.title('Dice Coefficient')

plt.subplot(1, 2, 2)
plt.plot(history.history['mean_iou'], label='Training Mean IoU')
plt.plot(history.history['val_mean_iou'], label='Validation Mean IoU')
plt.legend()
plt.grid()
plt.title('Mean IoU')

plt.show()
```



```
In [54]: def visualize_segmentation_with_cropping(image, predicted_mask, ground_truth_mask):
        """
        Visualizes the original image, predicted mask, ground truth mask, and segmented

        Args:
            image (array): Original image.
            predicted_mask (array): Predicted mask.
            ground_truth_mask (array): Ground truth mask.
        """
        # Threshold the predicted mask to create a binary mask for segmentation
        binary_mask = (predicted_mask > 0.5).astype(np.uint8)

        # Plotting the original image, predicted mask, ground truth mask, and segmented
        plt.figure(figsize=(16, 4))

        plt.subplot(1, 4, 1)
        plt.imshow(image)
        plt.title('Original Image')

        plt.subplot(1, 4, 2)
        plt.imshow(predicted_mask, cmap='gray')
        plt.title('Predicted Mask')

        plt.subplot(1, 4, 3)
        plt.imshow(ground_truth_mask, cmap='gray')
        plt.title('Ground Truth Mask')

        # Apply the binary mask to the original image to create the segmented output
        segmented_image = cv2.merge((binary_mask, binary_mask, binary_mask)) * image

        plt.subplot(1, 4, 4)
        plt.imshow(segmented_image)
        plt.title('Segmented Image')

        plt.show()

        # Example usage
        sample_image = X_val[0]
```

```

sample_ground_truth = y_val[0]

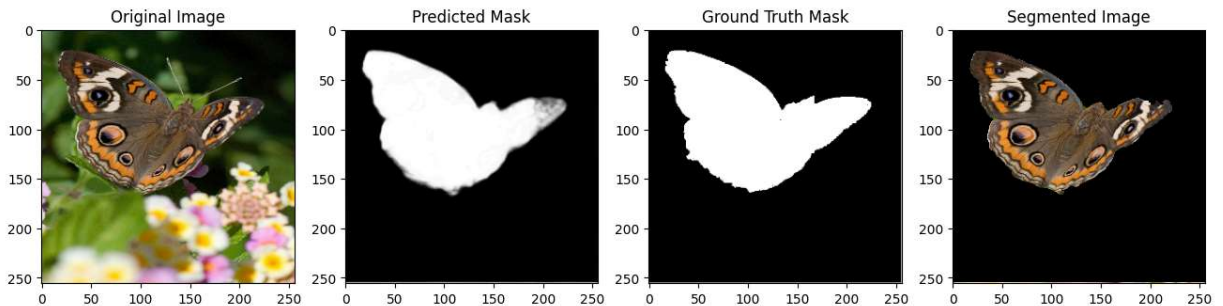
predicted_mask, _ = predict_single_image(sample_image, unet_model)

# Visualize the segmentation results with cropping
visualize_segmentation_with_cropping(sample_image, predicted_mask, sample_ground_tr

```

/usr/local/lib/python3.11/dist-packages/keras/src/models/functional.py:237: UserWarning: The structure of `inputs` doesn't match the expected structure.
Expected: ['keras_tensor_76']
Received: inputs=Tensor(shape=(1, 256, 256, 3))
warnings.warn(msg)

1/1 ————— 2s 2s/step



```

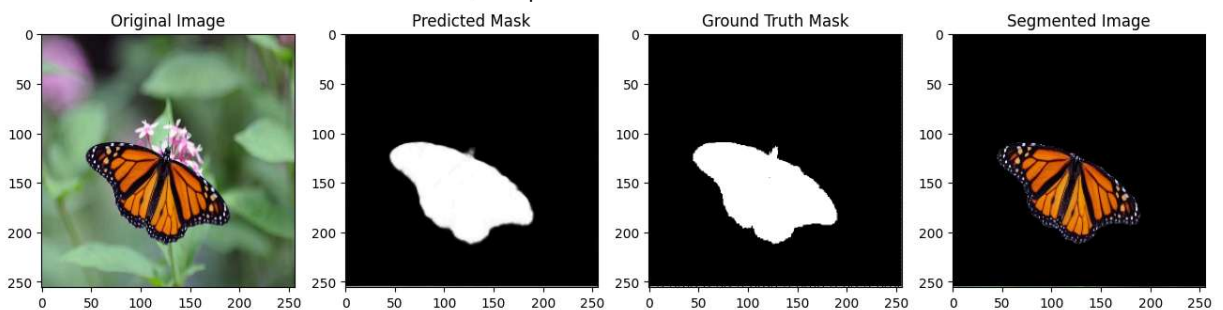
In [55]: # Example usage
sample_image = X_val[10]
sample_ground_truth = y_val[10]

predicted_mask, _ = predict_single_image(sample_image, unet_model)

# Visualize the segmentation results with cropping
visualize_segmentation_with_cropping(sample_image, predicted_mask, sample_ground_tr

```

1/1 ————— 0s 39ms/step



```

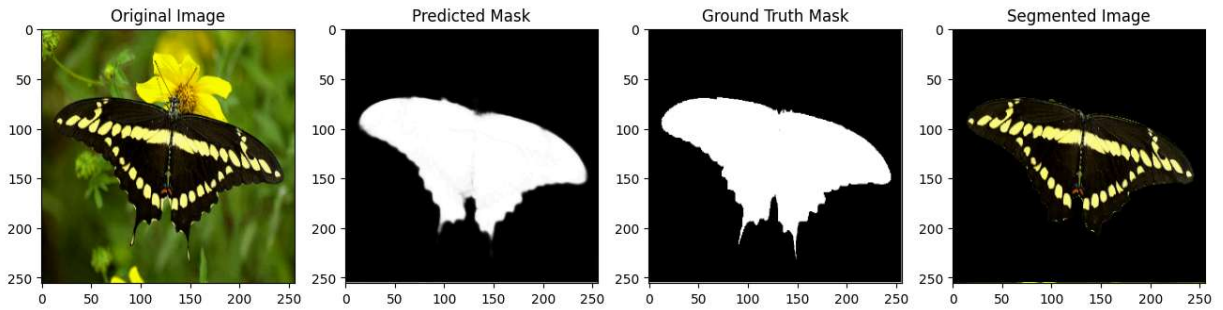
In [56]: # Example usage
sample_image = X_val[20]
sample_ground_truth = y_val[20]

predicted_mask, _ = predict_single_image(sample_image, unet_model)

# Visualize the segmentation results with cropping
visualize_segmentation_with_cropping(sample_image, predicted_mask, sample_ground_tr

```

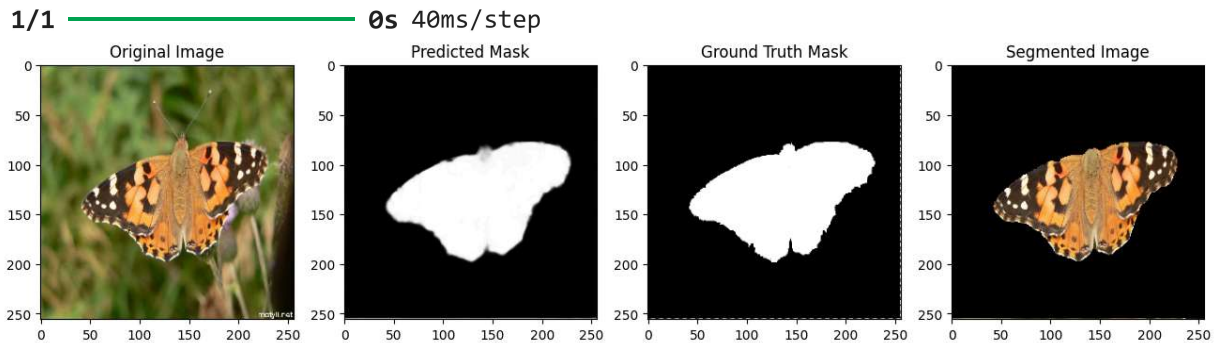
1/1 ————— 0s 36ms/step



```
In [57]: # Example usage
sample_image = X_val[50]
sample_ground_truth = y_val[50]

predicted_mask, _ = predict_single_image(sample_image, unet_model)

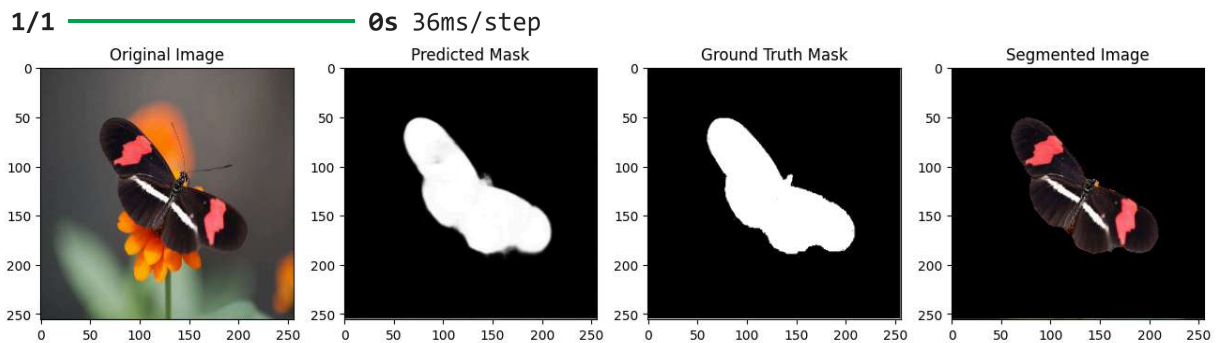
# Visualize the segmentation results with cropping
visualize_segmentation_with_cropping(sample_image, predicted_mask, sample_ground_tr
```



```
In [58]: # Example usage
sample_image = X_val[28]
sample_ground_truth = y_val[28]

predicted_mask, _ = predict_single_image(sample_image, unet_model)

# Visualize the segmentation results with cropping
visualize_segmentation_with_cropping(sample_image, predicted_mask, sample_ground_tr
```



```
In [59]: unet_model.save('final_UNET_Butterfly_Segmentation.keras')
```

```
In [ ]:
```